Keycloak — Meru App

Servicio de identidad (autenticación y autorización) usado por el backend de Meru App. Corre como contenedor aparte, independiente del backend Spring Boot.

Cómo levantarlo en desarrollo

1. Copiar `.env.example` a `.env` y completar las contraseñas (el `.env` real **no** se sube al repo).
2. Desde esta carpeta:

```
bash
docker compose up -d
```

3. Consola de administración: http://localhost:8081 (usuario/clave definidos en `.env`).
4. Si existe un export en `realm-export/meru-realm.json`, Keycloak lo importa automáticamente al arrancar (por el flag `--import-realm`). Si es la primera vez que configuran el realm, van a crearlo manualmente desde la consola (ver abajo) y luego lo exportan para versionarlo.

   Nota: se usa el puerto **8081** (no 8080) para no chocar con el backend Spring Boot, que suele correr en 8080.

Configuración inicial del realm (solo la primera vez)

1. Crear un realm nuevo: `meru-app`.
2. Crear los roles del realm: `admin`, `profesor`, `alumno`.
3. Crear un client para el frontend (tipo público, con PKCE) y otro para el backend si hace falta validar vía introspección en lugar de JWT firmado.
4. Configurar el mapper de roles para que viajen en el claim `realm_access.roles` del JWT (es el comportamiento por defecto).
5. Crear un usuario de prueba por cada rol para testear el login end-to-end.

Exportar el realm para versionarlo

Cada vez que cambien la configuración del realm (nuevo rol, nuevo client, etc.), exporten y commiteen el resultado:

```
bash
```

```
docker exec meru-keycloak /opt/keycloak/bin/kc.sh export \
 --dir /opt/keycloak/data/import \
 --realm meru-app \
 --users realm_file
```

Esto sobreescribe `realm-export/meru-realm.json` (o similar), que el resto del equipo importa automáticamente la próxima vez que levanten el compose desde cero.

Backend (Spring Boot) — cómo se conecta

El backend actúa como **resource server**: no gestiona contraseñas ni emite tokens, solo valida el JWT firmado por Keycloak contra su endpoint JWKS público. Configuración típica en `application.yml`:

```
yaml
spring:
 security:
  oauth2:
   resourceserver:
    jwt:
     issuer-uri: http://localhost:8081/realms/meru-app
```

Despliegue (a definir hacia la entrega final)

Para producción hay dos caminos principales:

- **Self-host en un servicio que soporte contenedores Docker** (Railway lo soporta directamente desde este mismo `docker-compose.yml`, adaptado a variables de entorno del proveedor; Render requiere un "Web Service" de tipo Docker por contenedor, sin compose nativo, por lo que Keycloak y su base de datos se configurarían como dos servicios separados).
- **Usar una base de datos administrada en lugar del contenedor Postgres local** — por ejemplo, otra base en Supabase dedicada solo a Keycloak (schema o proyecto separado del que usa el backend, para no mezclar datos de identidad con datos de negocio).

Antes de decidir el proveedor final, verificar el consumo de RAM de Keycloak contra los límites del plan gratuito elegido — es la causa más común de que el contenedor no arranque o tenga cold starts largos en tiers free.